

GIỚI THIỆU

Introduction

Trong bài lab này, em thực hiện xây dựng ứng dụng ASP.NET Web Forms theo mô hình kiến trúc 3 lớp (3-Tier Architecture), gồm:

- Data Access Layer (DAL)
- Business Logic Layer (BLL)
- Presentation Layer (WebUI)

Mục tiêu của bài lab là tìm hiểu cách tổ chức project theo mô hình 3 lớp và thực hiện các chức năng CRUD (Create, Read, Update, Delete) với SQL Server.

Chụp hình:

- Giao diện Solution Explorer sau khi tạo 3 project:
 - DAL
 - BLL
 - WebUI

EXERCISE 1 — XÂY DỰNG 3-TIER ARCHITECTURE

Bước 1 — Tạo Solution và các Project

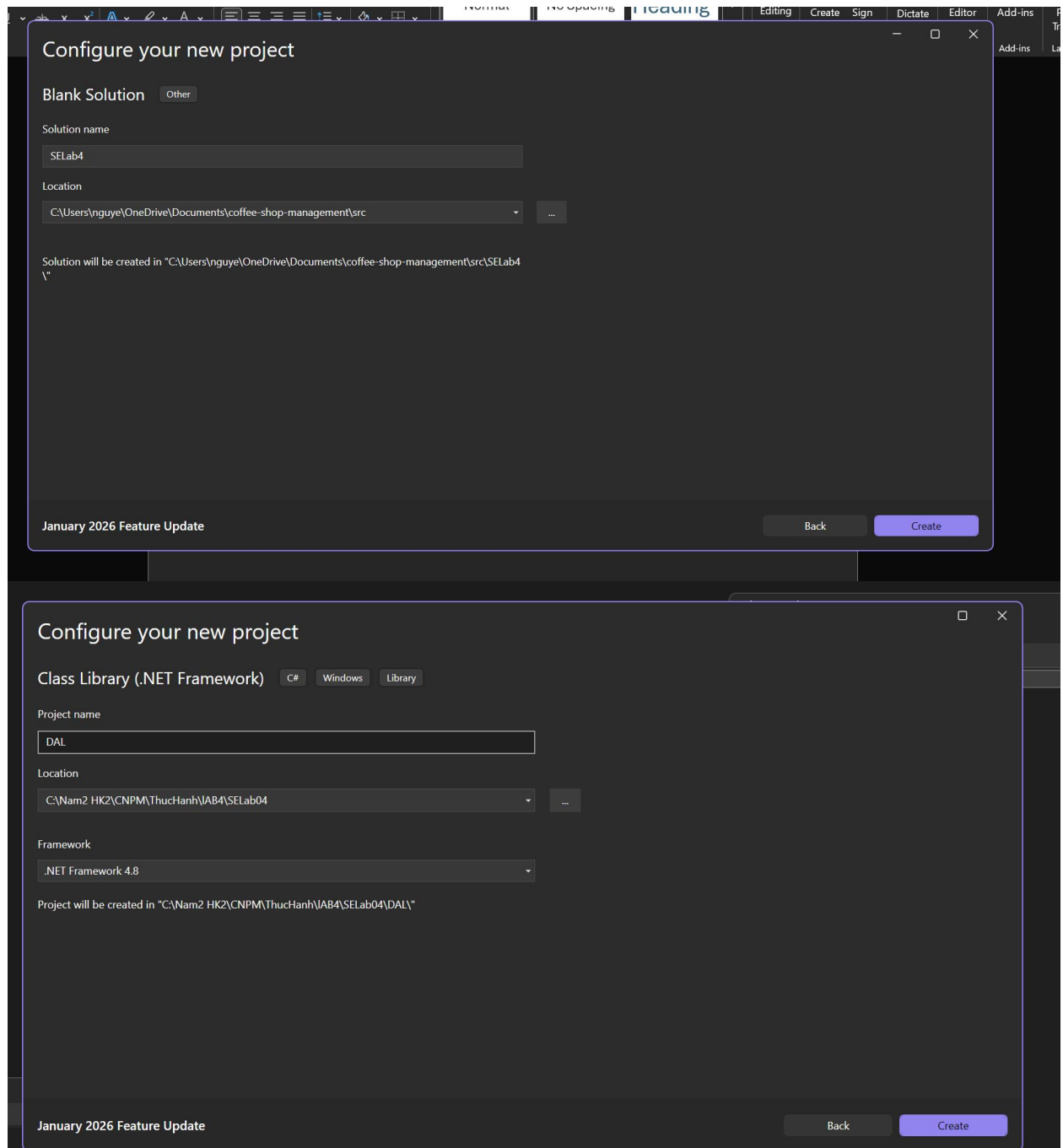
Nội dung

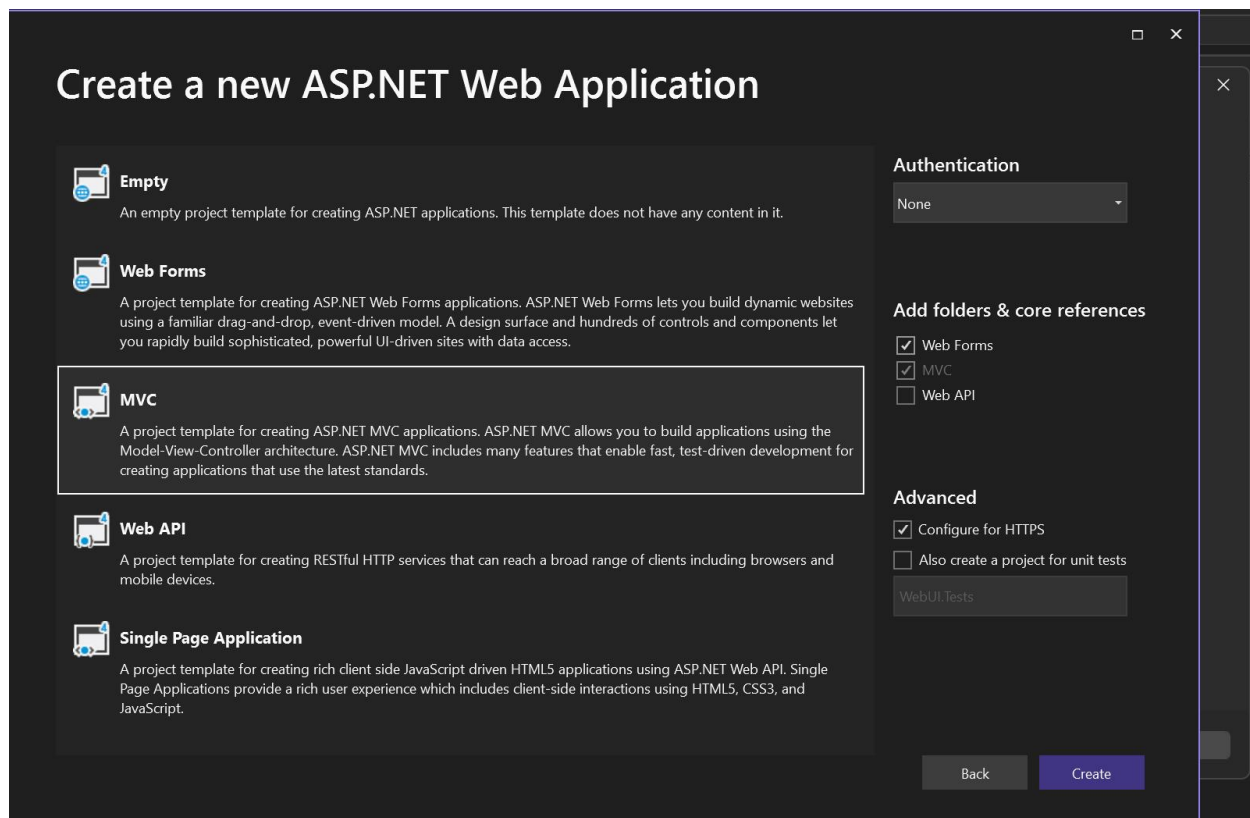
Đầu tiên, em tạo một Blank Solution tên SELab4 trong Visual Studio.

Sau đó thêm 3 project vào solution gồm:

- DAL (Class Library)
- BLL (Class Library)
- WebUI (ASP.NET Web Forms)

Việc tách project giúp chương trình dễ quản lý và đúng theo mô hình 3 lớp.





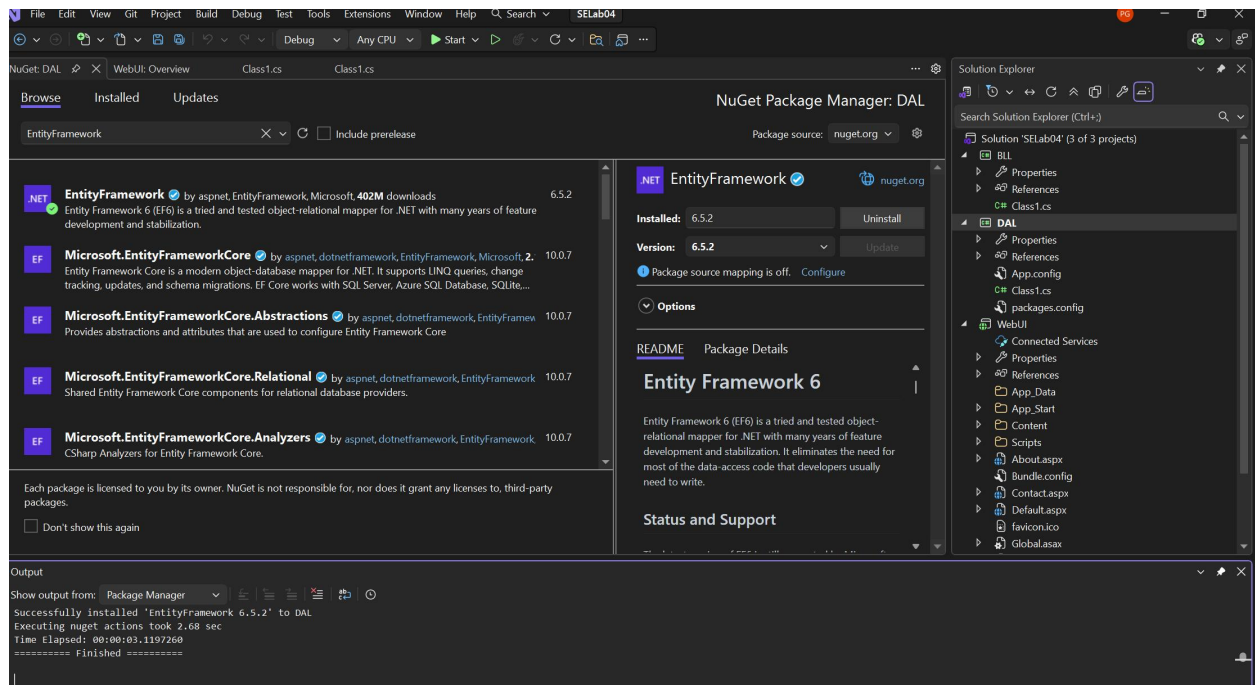
Bước 2 — Cài đặt Entity Framework

Nội dung

Tiếp theo, em cài đặt Entity Framework bằng NuGet Package Manager cho:

- DAL
- WebUI

Entity Framework giúp kết nối chương trình với cơ sở dữ liệu SQL Server dễ dàng hơn.



Bước 3 — Tạo Database và Table

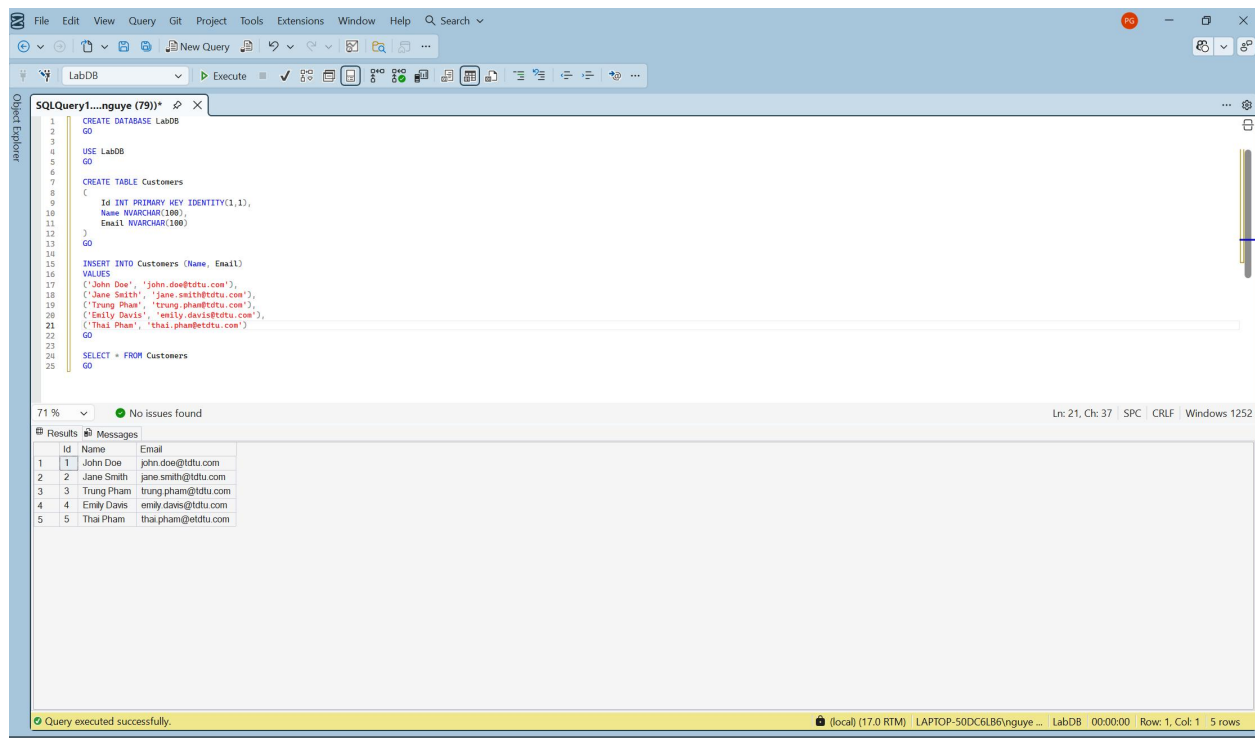
Nội dung

Trong SQL Server Management Studio, em tạo database tên LabDB.

Sau đó tạo bảng Customers gồm các cột:

- Id
- Name
- Email

Tiếp tục thêm dữ liệu mẫu vào bảng để kiểm tra chương trình.



Bước 4 — Tạo các Class trong DAL

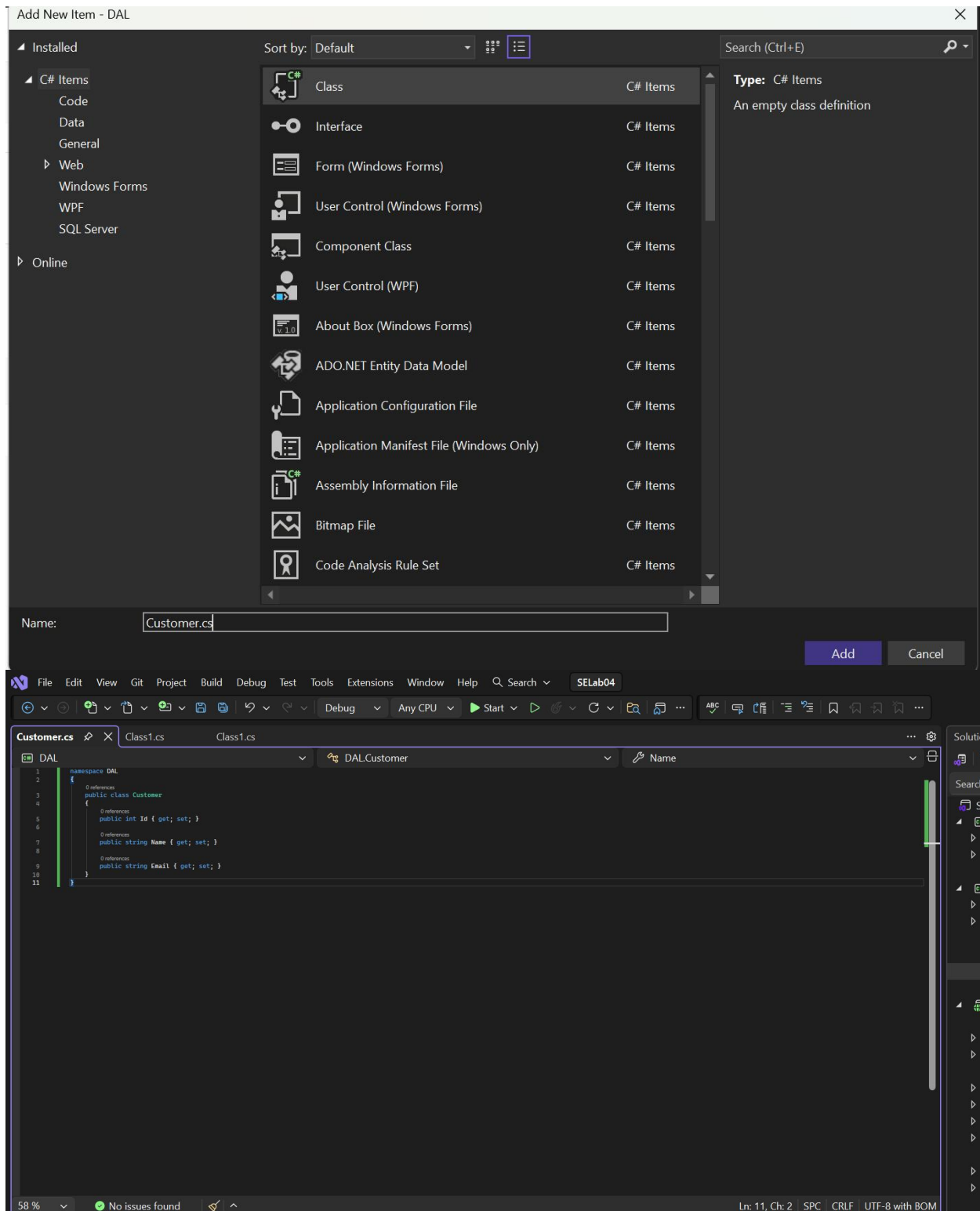
Nội dung

Trong project DAL, em tạo các class:

- Customer.cs
- CustomerRepository.cs
- Exer1DbContext.cs

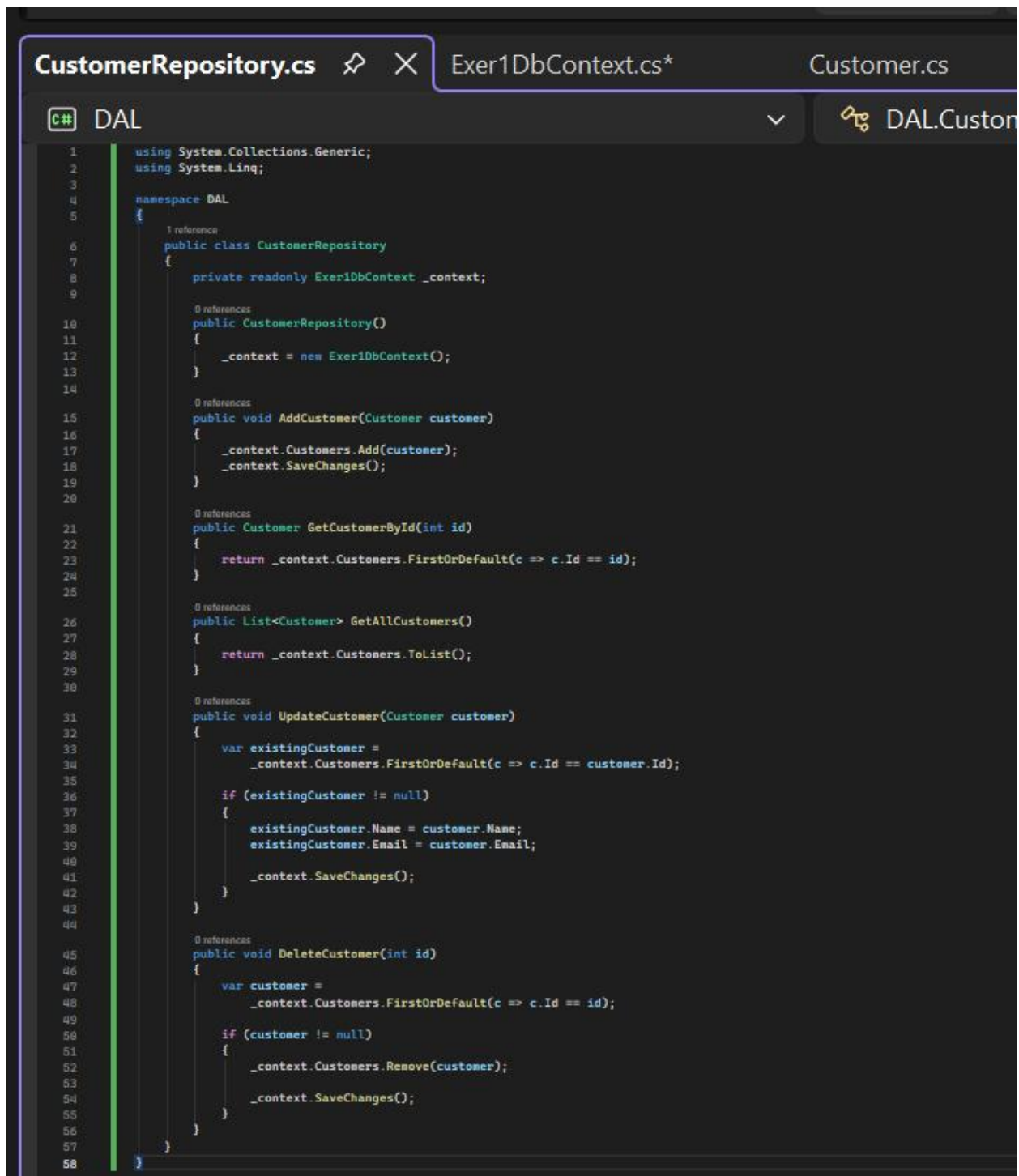
Các class này có nhiệm vụ:

- lưu thông tin khách hàng,
- kết nối database,
- thực hiện các thao tác CRUD.



The screenshot shows the Visual Studio IDE with the file `Exer1DbContext.cs` open. The code defines a namespace `DAL` containing a class `Exer1DbContext` that inherits from `DbContext`. The class has a `Customers` property of type `DbSet<Customer>` and a constructor that takes a connection string `"name=MyConn"`. The interface `DALExer1DbContext` is also visible in the Solution Explorer.

```
1 using System.Data.Entity;
2
3 namespace DAL
4 {
5     1 reference
6     public class Exer1DbContext : DbContext
7     {
8         0 references
9         public DbSet<Customer> Customers { get; set; }
10
11         0 references
12         public Exer1DbContext() : base("name=MyConn")
13         {
14         }
15     }
16 }
```

```
1 using System.Collections.Generic;
2 using System.Linq;
3
4 namespace DAL
5 {
6     1 reference
7     public class CustomerRepository
8     {
9         private readonly Exer1DbContext _context;
10
11         0 references
12         public CustomerRepository()
13         {
14             _context = new Exer1DbContext();
15         }
16
17         0 references
18         public void AddCustomer(Customer customer)
19         {
20             _context.Customers.Add(customer);
21             _context.SaveChanges();
22         }
23
24         0 references
25         public Customer GetCustomerById(int id)
26         {
27             return _context.Customers.FirstOrDefault(c => c.Id == id);
28         }
29
30         0 references
31         public List<Customer> GetAllCustomers()
32         {
33             return _context.Customers.ToList();
34         }
35
36         0 references
37         public void UpdateCustomer(Customer customer)
38         {
39             var existingCustomer =
40                 _context.Customers.FirstOrDefault(c => c.Id == customer.Id);
41
42             if (existingCustomer != null)
43             {
44                 existingCustomer.Name = customer.Name;
45                 existingCustomer.Email = customer.Email;
46                 _context.SaveChanges();
47             }
48         }
49
50         0 references
51         public void DeleteCustomer(int id)
52         {
53             var customer =
54                 _context.Customers.FirstOrDefault(c => c.Id == id);
55
56             if (customer != null)
57             {
58                 _context.Customers.Remove(customer);
59                 _context.SaveChanges();
60             }
61         }
62     }
63 }
```

Bước 5 — Tạo Business Logic Layer

Nội dung

Trong project BLL, em tạo class CustomerService.cs.

Lớp này dùng để xử lý nghiệp vụ và làm trung gian giữa WebUI và DAL.

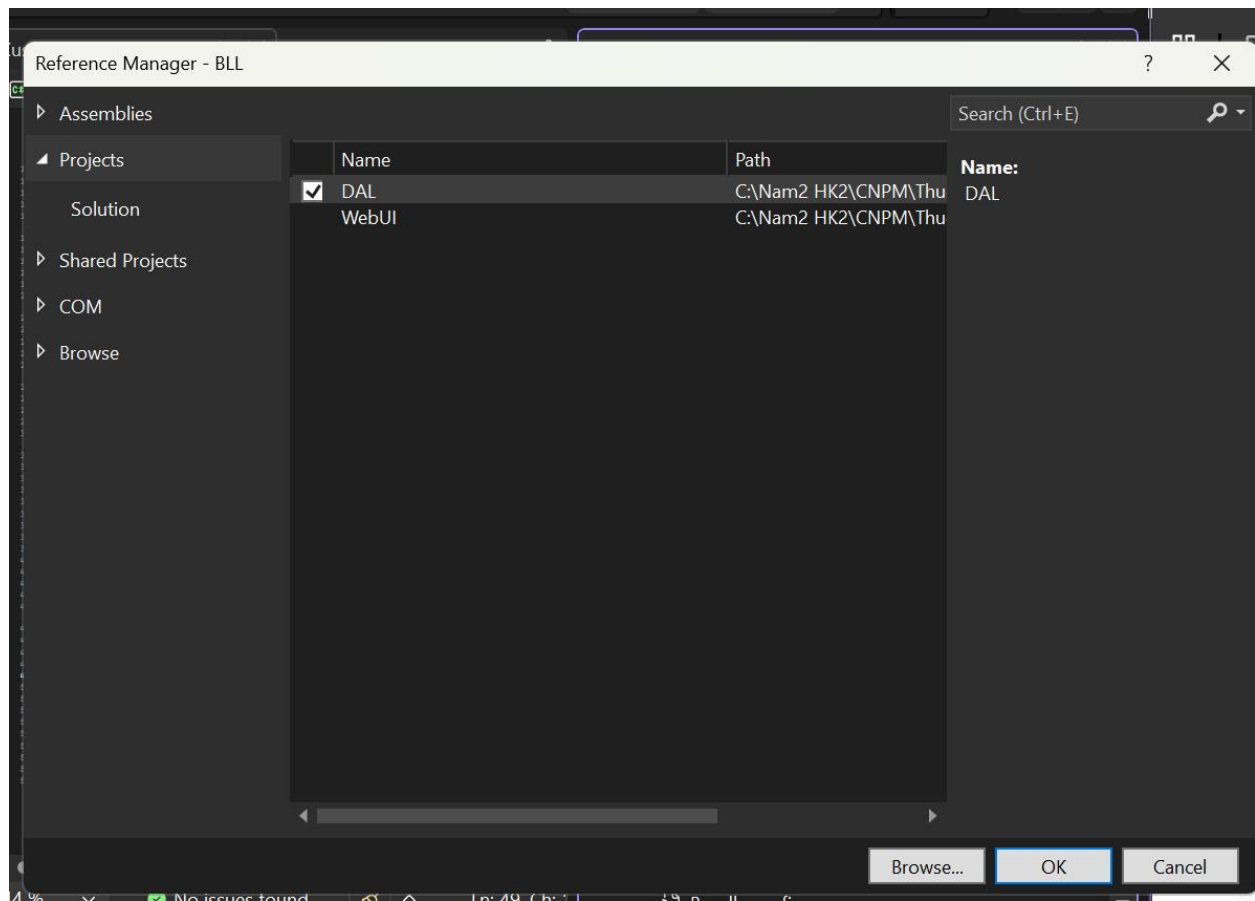
Bước 6 — Thêm Reference giữa các Project

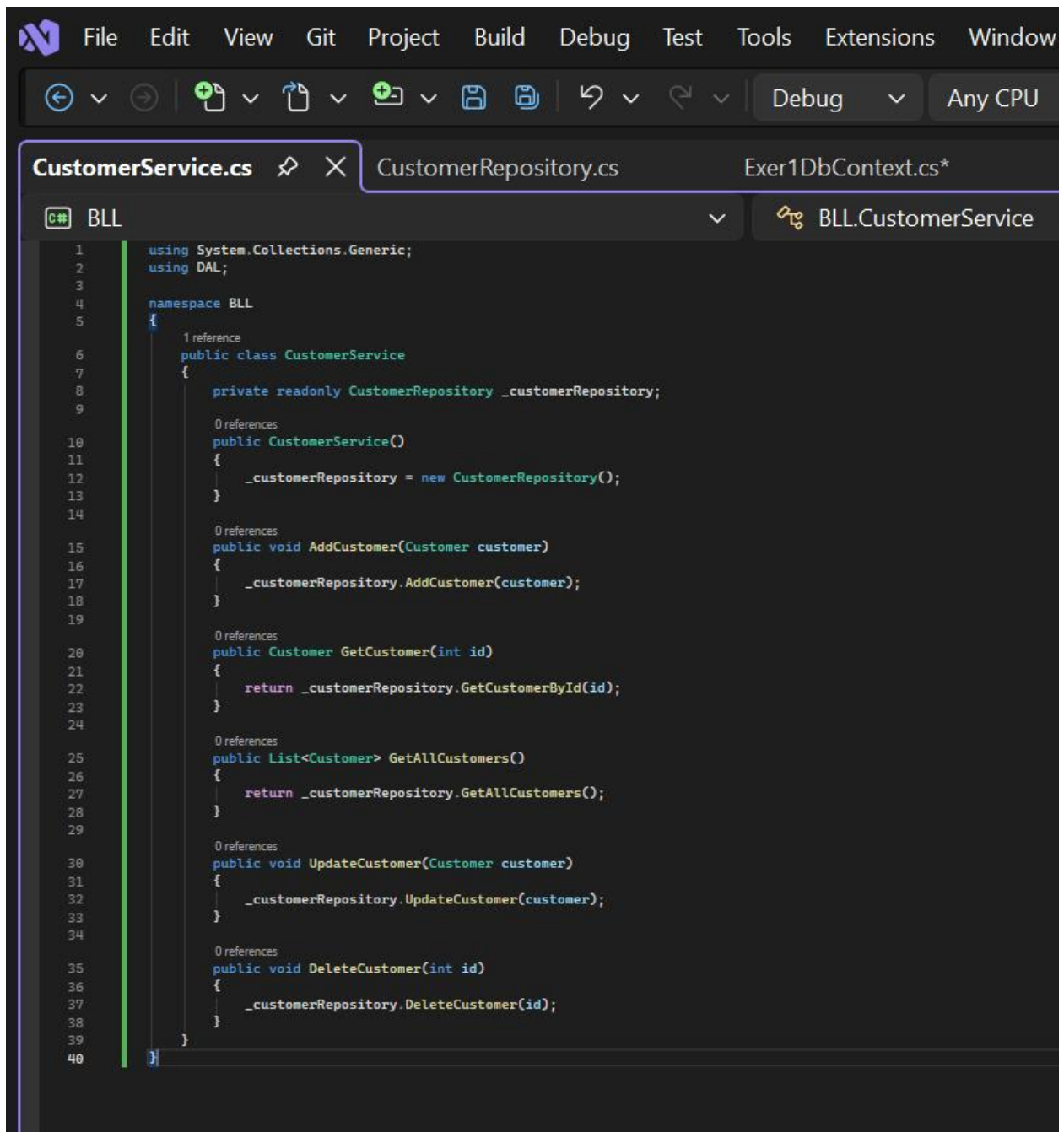
Nội dung

Tiếp theo, em thêm Reference:

- BLL tham chiếu DAL
- WebUI tham chiếu DAL và BLL

Điều này giúp các project có thể sử dụng class của nhau.

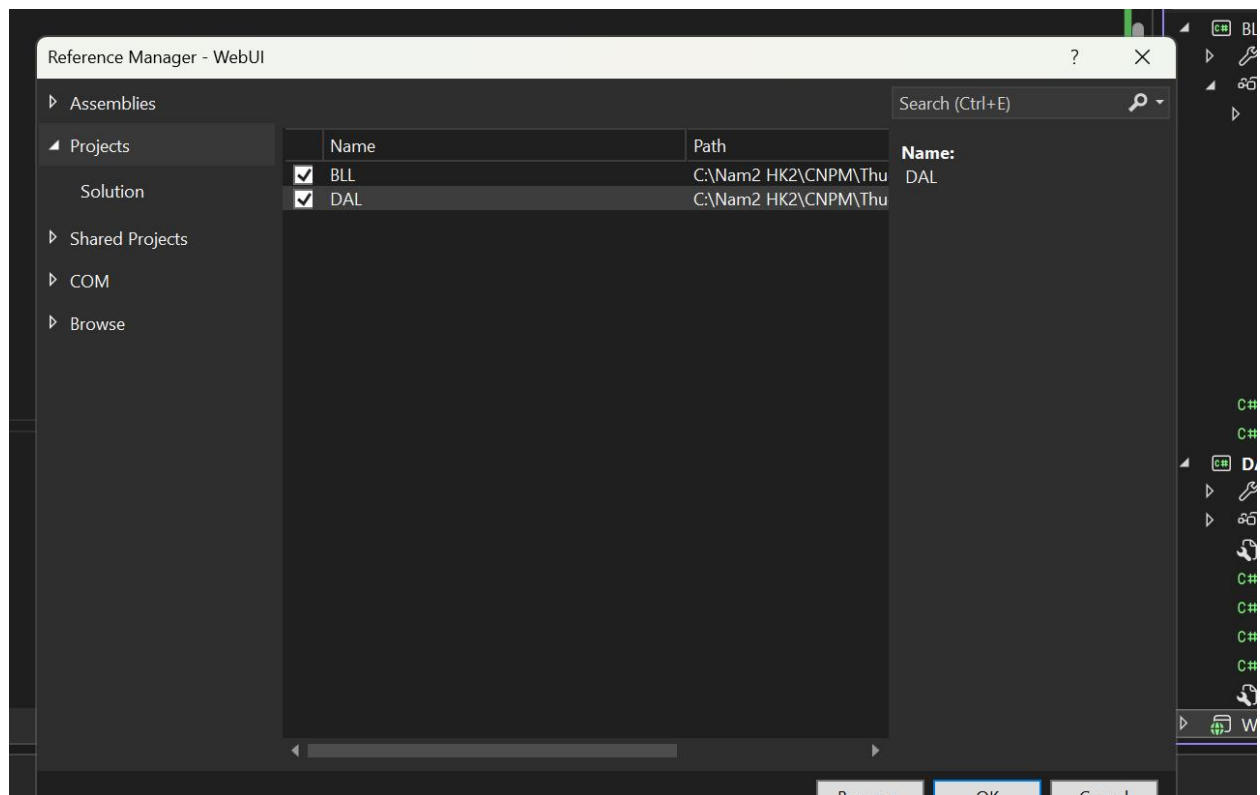




The image shows a Visual Studio window with the following components:

- Menu Bar:** File, Edit, View, Git, Project, Build, Debug, Test, Tools, Extensions, Window.
- Toolbar:** Includes icons for undo, redo, add, remove, save, and other standard development actions.
- Tab Bar:** Shows three open files: **CustomerService.cs** (active), **CustomerRepository.cs**, and **Exer1DbContext.cs***.
- Editor:** Displays the code for **BLL** (Business Logic Layer) in **CustomerService.cs**. The code is as follows:

```
1 using System.Collections.Generic;
2 using DAL;
3
4 namespace BLL
5 {
6     1 reference
7     public class CustomerService
8     {
9         private readonly CustomerRepository _customerRepository;
10
11         0 references
12         public CustomerService()
13         {
14             _customerRepository = new CustomerRepository();
15         }
16
17         0 references
18         public void AddCustomer(Customer customer)
19         {
20             _customerRepository.AddCustomer(customer);
21         }
22
23         0 references
24         public Customer GetCustomer(int id)
25         {
26             return _customerRepository.GetCustomerById(id);
27         }
28
29         0 references
30         public List<Customer> GetAllCustomers()
31         {
32             return _customerRepository.GetAllCustomers();
33         }
34
35         0 references
36         public void UpdateCustomer(Customer customer)
37         {
38             _customerRepository.UpdateCustomer(customer);
39         }
40
41         0 references
42         public void DeleteCustomer(int id)
43         {
44             _customerRepository.DeleteCustomer(id);
45         }
46     }
47 }
```



Bước 7 — Thiết kế giao diện Web Form

Nội dung

Em tạo Web Form tên CustomerWForm.aspx.

Giao diện gồm:

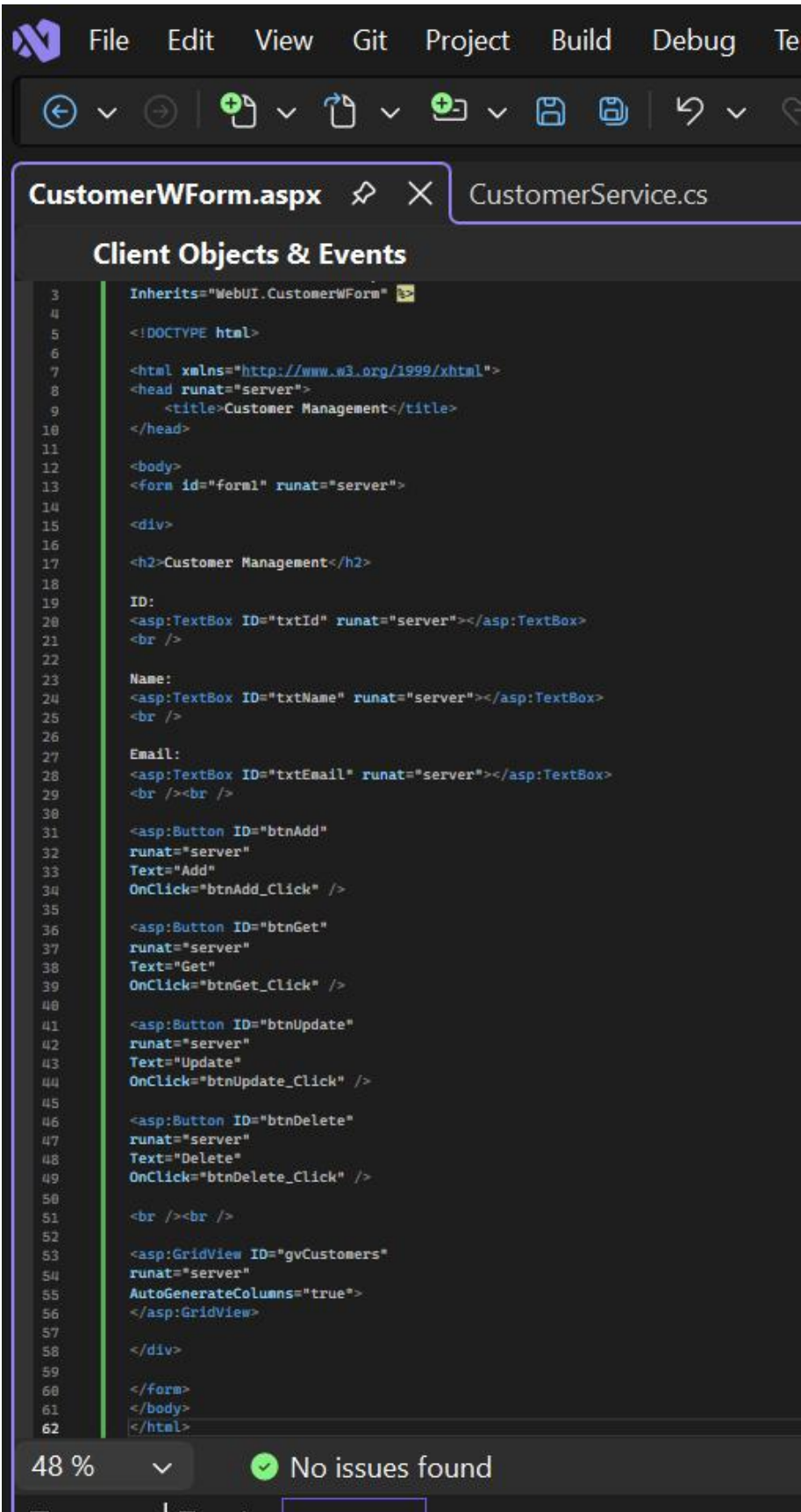
- TextBox nhập Id
- TextBox nhập Name
- TextBox nhập Email
- Button Add
- Button Get
- Button Update
- Button Delete
- GridView hiển thị danh sách khách hàng

Bước 8 — Xử lý CRUD trong Code-Behind

Nội dung

Trong file CustomerWForm.aspx.cs, em viết các hàm xử lý:

- thêm khách hàng,
- tìm khách hàng theo Id,
- cập nhật thông tin,
- xóa khách hàng,
- hiển thị danh sách khách hàng lên GridView.



CustomerWForm.aspx.cs* X CustomerWForm.aspx

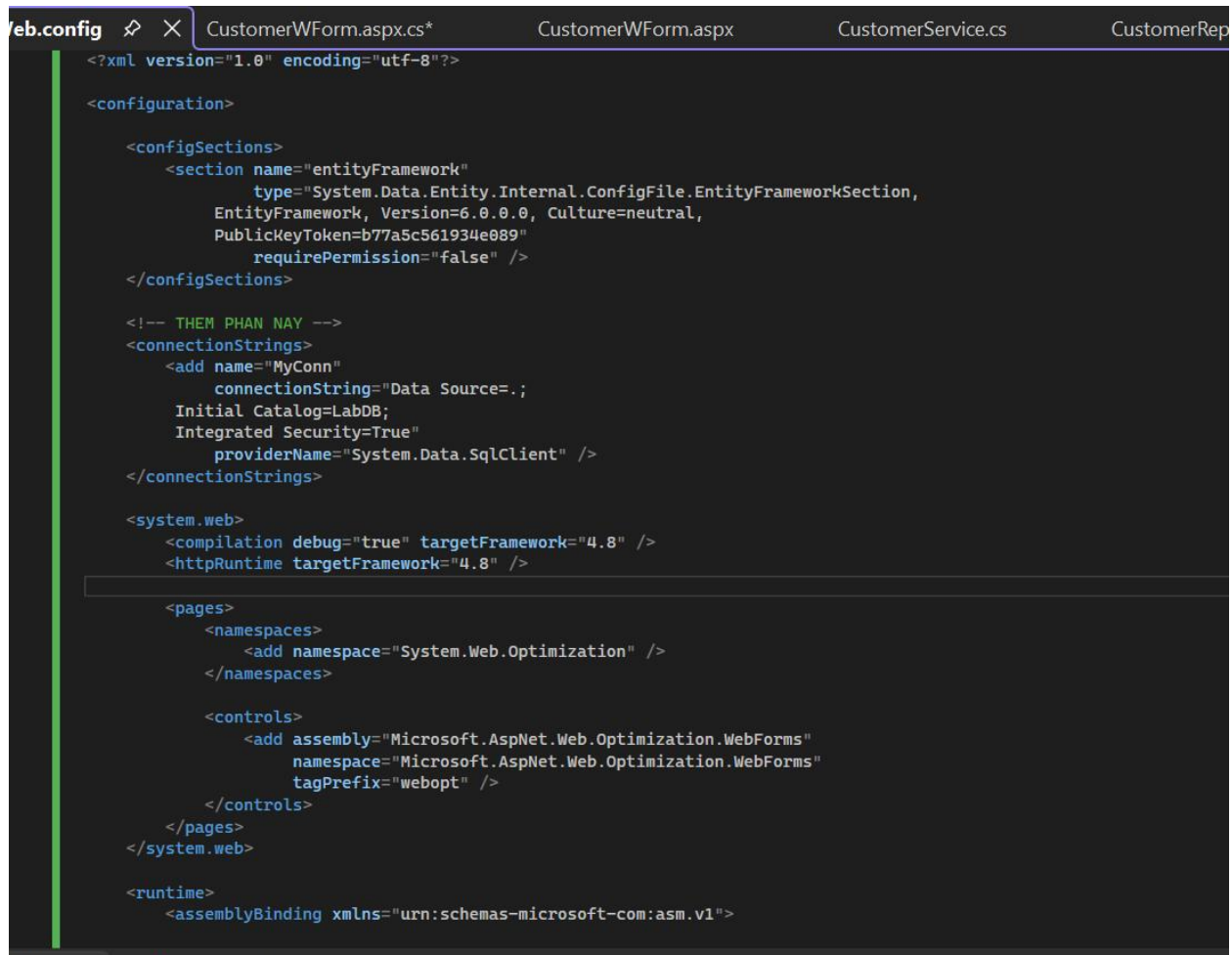
WebUI

```
1 using DAL;
2 using BLL;
3 using DAL;
4 using System;
5
6 namespace WebUI
7 {
8     public partial class CustomerWForm : System.Web.UI.Page
9     {
10         private readonly CustomerService _customerService =
11             new CustomerService();
12
13         protected void Page_Load(object sender, EventArgs e)
14         {
15             if (!IsPostBack)
16             {
17                 BindCustomerGrid();
18             }
19         }
20
21         private void BindCustomerGrid()
22         {
23             var customers = _customerService.GetAllCustomers();
24             gvCustomers.DataSource = customers;
25             gvCustomers.DataBind();
26         }
27
28         protected void btnAdd_Click(object sender, EventArgs e)
29         {
30             var customer = new Customer
31             {
32                 Name = txtName.Text,
33                 Email = txtEmail.Text
34             };
35             _customerService.AddCustomer(customer);
36             BindCustomerGrid();
37         }
38
39         protected void btnGet_Click(object sender, EventArgs e)
40         {
41             int id = int.Parse(txtId.Text);
42             var customer = _customerService.GetCustomer(id);
43             if (customer != null)
44             {
45                 txtName.Text = customer.Name;
46                 txtEmail.Text = customer.Email;
47             }
48         }
49
50         protected void btnUpdate_Click(object sender, EventArgs e)
51         {
52             var customer = new Customer
53             {
54                 Id = int.Parse(txtId.Text),
55                 Name = txtName.Text,
56                 Email = txtEmail.Text
57             };
58             _customerService.UpdateCustomer(customer);
59             BindCustomerGrid();
60         }
61
62         protected void btnDelete_Click(object sender, EventArgs e)
63         {
64             int id = int.Parse(txtId.Text);
65             _customerService.DeleteCustomer(id);
66             BindCustomerGrid();
67         }
68     }
69 }
```

Bước 9 — Cấu hình web.config

Nội dung

Em thêm connection string vào file web.config để kết nối ứng dụng với SQL Server



```
<?xml version="1.0" encoding="utf-8"?>

<configuration>

  <configSections>
    <section name="entityFramework"
      type="System.Data.Entity.Internal.ConfigFile.EntityFrameworkSection,
      EntityFramework, Version=6.0.0.0, Culture=neutral,
      PublicKeyToken=b77a5c561934e089"
      requirePermission="false" />
  </configSections>

  <!-- THEM PHAN NAY -->
  <connectionStrings>
    <add name="MyConn"
      connectionString="Data Source=.;
      Initial Catalog=LabDB;
      Integrated Security=True"
      providerName="System.Data.SqlClient" />
  </connectionStrings>

  <system.web>
    <compilation debug="true" targetFramework="4.8" />
    <httpRuntime targetFramework="4.8" />

    <pages>
      <namespaces>
        <add namespace="System.Web.Optimization" />
      </namespaces>

      <controls>
        <add assembly="Microsoft.AspNet.Web.Optimization.WebForms"
          namespace="Microsoft.AspNet.Web.Optimization.WebForms"
          tagPrefix="webopt" />
      </controls>
    </pages>
  </system.web>

  <runtime>
    <assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
```

Bước 10 — Chạy chương trình

Nội dung

Sau khi hoàn thành các bước trên, em đặt WebUI làm Startup Project và chạy chương trình bằng phím F5.

Chương trình chạy thành công và thực hiện được đầy đủ các chức năng CRUD.

Customer Management

Hỏi Gemini

localhost:44382/CustomerWForm

Thư mục mớiThẻ mới320 Câu hỏi trắc ng...GitHubAWS Console - Sign...

Customer Management

ID:

Name:

Email:

AddGetUpdateDelete

Id	Name	Email
1	John Doe	john.doe@tdtu.com
2	Jane Smith	jane.smith@tdtu.com
3	Trung Pham	trung.pham@tdtu.com
4	Emily Davis	emily.davis@tdtu.com
5	Thai Pham	thai.pham@tdtu.com